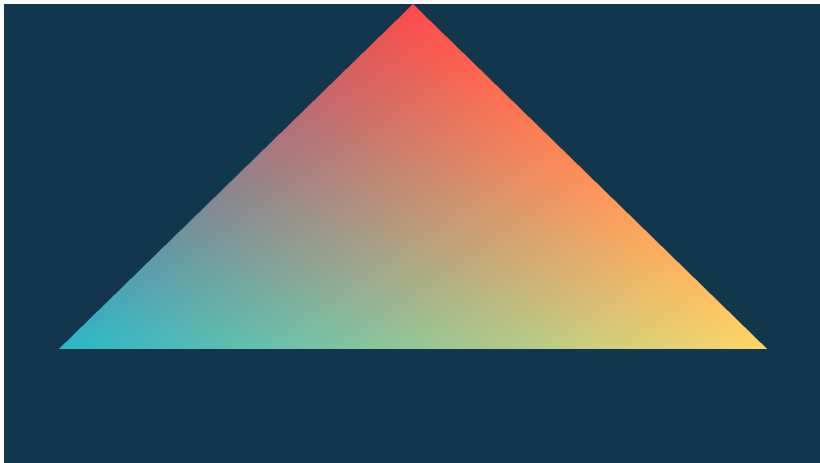


Компьютерная графика

Практика 2: Вращаем треугольник

2026



Задание 1

Уменьшим треугольник в 2 раза, используя **immediate** значение

- В шейдере заводим `var<immediate> scale: f32;` и домножаем где-то координаты на неё
- В коде инициализации, при создании **render pipeline** надо создать **pipeline layout**, выставить ему `immediate_size = 4`, и передать его в descriptor при создании **render pipeline**
 - C++: `WGPUPipelineLayoutDescriptor`,
`wgpuDeviceCreatePipelineLayout`
 - Rust: `device.create_pipeline_layout()`
- Перед вызовом рисования, передаём значение в **render pass** с `offset = 0`
 - C++: `wgpuRenderPassEncoderSetImmediates()`
 - Rust: `render_pass.set_immediates(), bytemuck::bytes_of()`

Задание 1



Задание 2

Добавим анимированное значение

- В шейдере нам нужны 2 immediate значения $\implies$ заводим под них структуру

```
struct Immediates {  
    scale: f32,  
    angle: f32,  
}  
var<immediate> immediates: Immediates;
```

- Обращаемся к её полям как `immediates.scale` и `immediates.angle`
- Нужно повернуть вектор координат на переданный угол (формулы смотри в лекции; нам нужны только координаты XY)

Задание 2

Добавим анимированное значение

- В создании **pipeline layout** обновляем `immediate_size = 8`
- В основном коде рисования заводим переменную `angle`, как-то зависящую от `time`
- Перед вызовом рисования, передаём её значение в **render pass** так же, как `scale`, но с `offset = 4`

Задание 2



Заменяем ручное преобразование на матрицу 4×4

- В шейдере заменяем `scale` и `angle` на одно поле `transform: mat4x4f` (саму структуру лучше оставить)
- Надо подумать, что надо домножить на эту матрицу и с какой стороны :)

Задание 3

Заменяем ручное преобразование на матрицу 4×4

- В создании **pipeline layout** обновляем `immediate_size = 64`
- Перед вызовом рисования заводим массив из 16-ти float'ов и заполняем матрицей поворота + масштабирования
- Должен остаться один вызов `set_immediates, offset = 0`

Добавляем в матрицу сдвиг, зависящий от времени

- Перед вызовом рисования заводим переменные x и y , как-то зависящие от `time`
- Вставляем их в нужное место матрицы

Задание 4



Задание 5

Исправляем “сплюснутость” треугольника: добавляем обработку aspect ratio экрана

- В структуру **Immediates** в шейдере добавляем ещё одну матрицу **view: mat4x4f**, она должна применяться *после* матрицы **transform**
- В создании **pipeline layout** обновляем **immediate_size = 128** – это максимум, поддерживаемый большинством платформ
- Перед вызовом рисования заводим ещё один массив из 16-ти float’ов и заполняем матрицей, которая делит координату X на **aspect_ratio**
- **aspect_ratio** вычисляется как **width / height** (важно, чтобы это было floating-point деление, а не целочисленное с округлением)

Задание 5



Задание 6*

Заменяем треугольник на управляемый шестиугольник

- Нужно поменять количество вершин в `render_pass.draw()`
- Нужно обновить захардкоженные массивы координат и цветов в шейдере
- Нужно сделать так, чтобы шестиугольник можно было двигать по обеим осям стрелками клавиатуры
 - В коде уже есть hash set `keydown/app.keydown`
 - C++: `keydown.contains(SDLK_LEFT)`, ...
 - Rust: `app.keydown.contains(&KeyCode::ArrowLeft)`, ...
- Сдвиг должен быть пропорционален `dt`

Задание 6*

